

Market Making with Reinforcement Learning

Project Report

Xinwu Yang

May 2, 2025

1 Project Goal

The aim of this project was to reproduce the NeurIPS 2019 study “*Reinforcement Learning for Market Making in a Multi-Agent Dealer Market*” [GVX⁺19]. Specifically, we wanted to

1. Build a simulator of a dealer market that supports market makers and stochastic investor flow;
2. Train a PPO (Proximal Policy Optimization) [SWD⁺17] agent to act as a market maker, learning spreads and hedging decisions from the simulation environments;
3. Benchmark the RL agent against a random baseline and a persistent baseline, and study the impact of key hyper-parameters (inventory risk penalty, hedge fraction, PPO clip ϵ , etc);

2 Environments

In this section, we will introduce our implementation of the simulator for the dealer market following Section 3 of [GVX⁺19]. We will first discuss the basic elements of the environment, and then we will proceed to discuss how the simulation works.

2.1 States

A state/observation is a 4-tuple $(v_{t-1}, z_{t-1}, P_t, S_{ref}(v_{t-1}))$ consisting of

- v_{t-1} : Number of transactions in the previous time step $t-1$. We assume that each investor will submit its order based on a Gamma distribution $Gamma(\alpha, \beta)$ with parameter α, β ;
- z_{t-1} : Inventory level at time $t-1$;
- P_t : Mid-price of the stock, determined by a Geometric Brownian motion

$$dP_t = \mu P_t dt + \sigma P_t dW_t, \quad (1)$$

where $\mu, \sigma > 0$ and W_t is a standard Brownian motion. It is well-known that (1) admits a closed-form solution

$$P_t = P_{t-1} \exp \left(\mu \Delta_t - \frac{1}{2} \sigma^2 \Delta_t + \sigma \sqrt{\Delta_t} Z \right), \quad (2)$$

where Δ_t is the time step, and $Z \sim N(0, 1)$.

- $S_{ref}(v_{t-1})$: Reference spread of the stock, given the current trading volume v_{t-1} . For simplicity, we assume that

$$S_{ref}(v) := AP_t + B\sqrt{v}P_t. \quad (3)$$

For our experiment, we set $A := 2 \times 10^{-4}$ and $B = 2 \times 10^{-5}$. For a more realistic setup, we should calibrate S_{ref} based on the shape of the limit order book (LOB). See Section 7.2 of [GVX⁺19].

2.2 Actions

An action is a 3-tuple $(\epsilon_a, \epsilon_b, x_t)$, where

- $\epsilon_a \in [-1, 1]$: Market maker will choose $S_a(t) := P_t + S_{ref}(v_{t-1})(1 + \epsilon_a)$ as the ask price
- $\epsilon_b \in [-1, 1]$: Market maker will choose $S_b(t) := P_t - S_{ref}(v_{t-1})(1 + \epsilon_b)$ as the bid price
- $x_t \in [0, 1]$: Fraction for hedging. Depending on the current inventory level z_t , the market maker will buy or sell $x_t z_t$ to hedge the inventory risk.

2.3 Rewards

In finance, we usually call the reward as PnL (Profit and Loss). In this report, we will use them interchangeably. The PnL for MM consists of three parts:

$$\text{PnL}(t) = \text{Spread PnL}(t) + (P_t - P_{t-1})z_{t-1} + x_t z_t P_{hedge}(t),$$

where

- Spread PnL(t) is given by accumulating all the shares bought and sold at time t ;
- $P_{hedge}(t)$ is determined by the sign of z_t , namely

$$P_{hedge}(t) := \begin{cases} S_b(t), & z_t > 0, \\ S_a(t), & z_t < 0. \end{cases}$$

2.4 Simulator

We describe how our simulator works when our agent interacts with the environment. At each time step t ,

1. The agent observes $(v_{t-1}, z_{t-1}, P_t, S_{ref}(v_{t-1}))$;
2. Based on the observation, take action $(\epsilon_a, \epsilon_b, x_t)$;
3. The investors submit their orders. The order size follows a Gamma distribution $\text{Gamma}(\alpha, \beta)$ with parameter α and β . The direction is buying or selling with equal probability;
4. The reward is computed for the agent, and we repeat steps 1-3.

3 One Market Maker Experiment

3.1 Experimental Settings

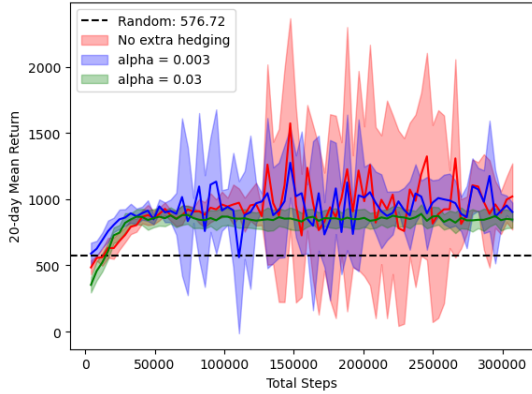
In our experiments, we mainly follow Section 5.3 of [GVX⁺19]. We use 2-layer fully-connected neural network with 256 nodes in each layer and $\tanh$ activation functions for PPO policy network, and a PPO clip parameter of 0.2. For training iteration, we apply one step minibatch SGD via Adam with batch size of 256, and the learning rate is set to be 3×10^{-4} .

For each experiment, we do 300 trajectory rollouts, and each rollout takes 1024 steps. After every 4 rollouts, we evaluate our policy for 520 steps (20 trading days, with 6.5 trading hours per day) for 10 times, and compute the mean and standard deviation.

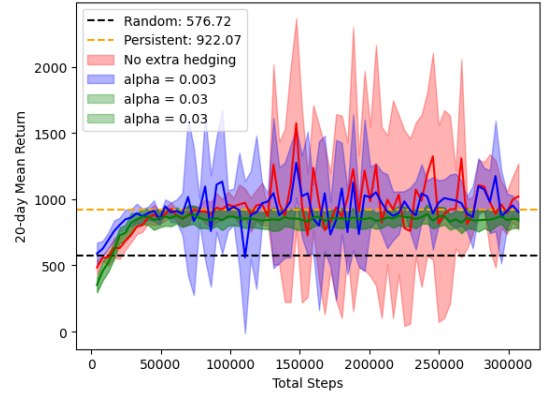
In the financial industry, high variance is not preferred. One major factor that brings in variance is the inventory. The inventory carried into the next time step is exposed to the potential large price change of the asset. To incorporate such risk-averse nature, we modify the PnL formulation by adding one more penalty term ¹:

$$\text{PnL}(t) = \text{Spread PnL}(t) + (P_t - P_{t-1})z_{t-1} + x_t z_t P_{\text{hedge}}(t) - \alpha(\text{Inventory PnL}(t))^2,$$

where we choose $\alpha := 0.03, 0.003$. The trainings can converge, and the evaluation graph is presented in Figure 1a.



(a) RL vs Random([-1,1])



(b) RL vs Random([-1,1]) vs Persistent(1,1,0.5)

Figure 1: Evaluation along training

3.2 Discussions

We made some comments based on our experiment results.

1. We could observe that, without adding the penalty term, the variation of the PnL is large; after adding the penalty term, the variance is mitigated. We could infer that much of the variation comes from the inventory.
2. Another observation for one market maker case is that, all the orders will go to the market maker; and on average, the size of buying orders should be the same as the size of the selling orders. Therefore, ideally, deterministically setting $(\epsilon_a, \epsilon_b) = (1, 1)$ can extract most of the spread PnL. To hedge the inventory risk, setting $x_t \equiv 0.5$ could reduce most of the inventory risk. This produces a persistent baseline, and yields satisfactory performance. The comparison is shown in Figure 1b.

4 Two Market Maker Case

In this section, we will detail our adjustments to incorporate the two-agent scenario and present our experiment results.

¹We remark that this is different from the presentation slides, which used Inventory PnL at $(t - 1)$ timestep. I don't have a good explanation why we should choose current or previous time step.

4.1 Environmental Adjustment

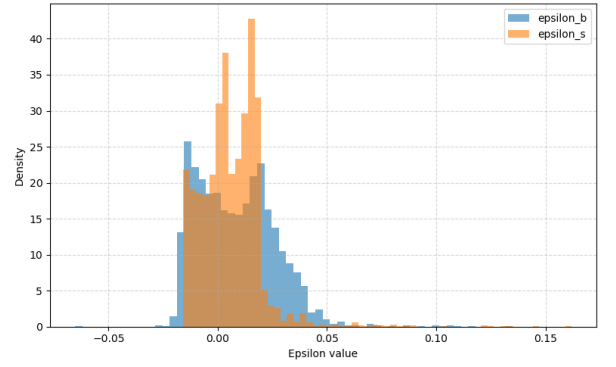
We need to adjust the investors' order flows in the two-agent case. When an investor submits its order (size, direction), it will pick a more favorable agent to execute the order. In particular, if it is a buy order, it will pick the agent with lower ask; conversely if it is a sell order, it will pick the agent with higher bid.

4.2 Experiments

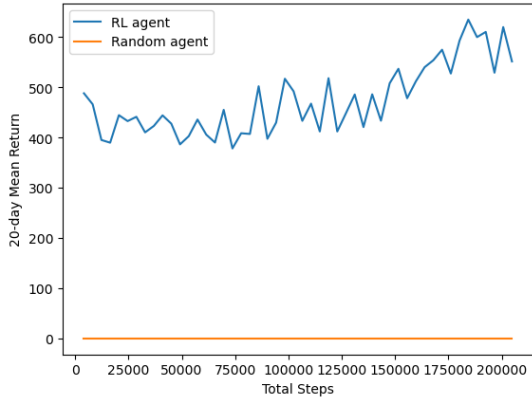
We changed our our network to be 3 layers of fully-connected neural networks with 256 nodes, and used *tanh* to be the activation function. We still do evaluation along the traing as in Subsection 3.1. After training, we also put the model in a rollout with 3000 steps, and collect the distribution of ϵ_b and ϵ_s .



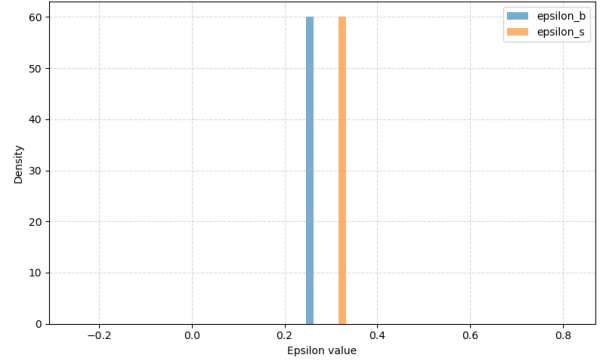
(a) RL vs Random(1,1,0.3)



(b) Epsilon density



(a) RL vs Persistent(0.5, 0.5, 0)



(b) Epsilon density

Figure 3: RL vs Persistent(0.5, 0.5, 0)

4.3 Discussions

Before discussing our experimental results, let us recall that, unlike the single-agent case, the more favorable market maker will capture the investors' orders. Meanwhile, we would like state [GVX⁺19, Theorem 1], which gives the optimal choice of (ϵ_s, ϵ_b) analytically.

Theorem 1 *The expected spread PnL of an agent competing with a market-maker that quotes buy/sell prices according to the distributions $F_j^{comp}(x) := \Pr[\epsilon_j^{comp} \leq x]$, $j \in \{b, s\}$, is maximized by setting*

$$\epsilon_j = \arg \max_{\epsilon} (1 + \epsilon) \left(1 - F_j^{comp}(\epsilon) + p \hat{F}_j^{comp}(\epsilon) \right),$$

where p is the probability that the agent wins the trade when both agents quote the same price, and $\hat{F}_j^{comp}(\epsilon) := \Pr[\epsilon_j^{comp} = \epsilon] = F_j^{comp}(\epsilon) - F_j^{comp}(\epsilon^-)$. The result holds under the assumption that investor trades are possibly random but independent of $\epsilon_b^{comp}, \epsilon_s^{comp}$.

In particular, if F_b^{comp} and F_s^{comp} are $\text{Unif}[a, b]$, $-1 \leq a \leq b$, then

$$\epsilon_b^* = \epsilon_s^* = \frac{1}{2} \max(2a, b - 1).$$

Now we give some comments on our experimental results.

1. If we look at Figure 2a and 3a, we do not see a clear improvement over the training. In fact, we didn't observe the convergence in the training.
2. For the RL vs Random case, by Theorem 1, we know that the optimal $\epsilon^* = 0$. From the epsilon density graph 2b, we see that the RL agent will choose ϵ near 0 with high probability.
3. For the RL vs Persistent case, by Theorem 1, the optimal $\epsilon^* = 0.5^-$ (i.e., some value smaller than 0.5). The intuition is that, the competitor is setting its $\epsilon \equiv 0.5$ deterministically. If we set some value lower than 0.5, we could take all the orders, and reduce to the single-agent case. Although our RL agent is able to take all the orders, it fails to set the optimal value of ϵ .

5 Future Works

1. We think that according to the model output in Section 4, it still needs further tuning. Aiming at the analytic result given by Theorem 1, we should try different network architecture and hyperparameters.
2. There are a few works applying Deep Learning or Reinforcement Learning methods on market making [BI19, BGW⁺25], and there are also traditional methods like [AS08, GLFT13]. We should benchmark our RL model with these related works, and investigate the performance.
3. In the Subsection 2.1, we mention that the mid-price process P_t follows a Geometric Brownian Motion with constant volatility factor σ . In practice, $\sigma = \sigma(t, P_t)$ depends on time and mid-price P_t . We could model $\sigma(t, P_t)$ using a more sophisticated model.

References

- [SWD⁺17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov, *Proximal policy optimization algorithms*, arXiv preprint arXiv:1707.06347 (2017).
- [AS08] Marco Avellaneda and Sasha Stoikov, *High-frequency trading in a limit order book*, Quantitative Finance **8** (2008), no. 3, 217–224.
- [BGW⁺25] Yahui Bai, Yuhe Gao, Runzhe Wan, Sheng Zhang, and Rui Song, *A review of reinforcement learning in financial applications*, Annual Review of Statistics and Its Application **12** (2025), no. 1, 209–232.

- [BI19] Taweh Beysolow II, *Market making via reinforcement learning*, Applied Reinforcement Learning with Python: With OpenAI Gym, Tensorflow, and Keras, Springer, 2019, pp. 77–94.
- [GLFT13] Olivier Guéant, Charles-Albert Lehalle, and Joaquin Fernandez-Tapia, *Dealing with the inventory risk: a solution to the market making problem*, Mathematics and financial economics **7** (2013), 477–507.
- [GVX⁺19] Sumitra Ganesh, Nelson Vadori, Mengda Xu, Hua Zheng, Prashant Reddy, and Manuela Veloso, *Reinforcement learning for market making in a multi-agent dealer market*, arXiv preprint arXiv:1911.05892 (2019).